2019 IEEE International Conference on Systems, Man and Cybernetics (SMC)
Bari, Italy. October 6-9, 2019

Coverage Optimization in the Cooperative Surveillance Task using Multiple Micro Aerial Vehicles

Matěj Petrlík[1], Vojtěch Vonásek[1], Martin Saska[1]

***Abstract*— In the task of cooperative surveillance using Micro Aerial Vehicles (MAVs), MAVs cooperatively observe a given set of Areas of Interest (AoI). The missions are usually prepared in a decoupled manner: first, the sensing locations are found, followed by computations of the trajectories assuming GPS-based localization. The precision of GPS may, however, be insufficient to keep the MAVs in compact groups, which may lead to mutual collisions. To avoid the collisions between MAVs, a camera-based on-board localization has to be used. This however requires to maintain positions of the team members in the given range to enable reliable on-board localization (each MAV has to be visible from other ones). The task of the mission planning is to find an appropriate distribution of MAVs above AoIs together with feasible trajectories from a depot to reach these locations. The on-board localization constraints and MAV motion constraints have to be satisfied during the entire mission. We propose a modification of RRT (Rapidly Exploring Random Tree) for this mission planning. The algorithm first explores the state space to find suitable sensing locations together with feasible trajectories towards them. Then, the sensing locations are optimized using Particle Swarm Optimization (PSO). The proposed method has been verified in numerous simulations and outdoor experiments. The achieved results exhibit significantly better performance in terms of lower computational power and complexity of solved scenarios than the state-of-the-art solutions.**

I. Introduction

Micro Aerial Vehicles (MAVs) have been used in various applications including field surveillance, crowd monitoring, and borders protection. In these tasks, MAVs are distributed in a way that each AoI (Area of Interest) is observed by a single MAV or multiple MAVs. The surveillance of the same area using closely cooperating MAVs brings additional information, as each MAV can observe the area from a different angle or altitude.

Current approaches for multi-MAV surveillance focus mainly on finding the sensing locations and rely on an open-access localization system like GPS to navigate the MAVs to these locations. However, the precision of GPS may be insufficient to keep precise positions of the MAVs, which may lead to mutual collisions [1]. This problem needs to be considered if compact MAV groups are required for the surveillance task with desired mutual distances lower than GNSS (Global Navigation Satellite System) accuracy, which is roughly 5-10 meters in open space areas. In urban environments, the accuracy may exceed even 20 meters due to signal reflections on walls of buildings.

[1]Authors are with the Dept. of Cybernetics, Faculty of Electrical Engineering, Czech Technical University in Prague, Technicka 2, 166 27, Prague 6, Czech Republic; Email: {matej.petrlik, vojtech.vonasek, martin.saska}@fel.cvut.cz.
The presented work has been supported by projects: GACR 19-22555Y, GACR 17-16900Y, CESNET LM2015042, SGS17/187/OHK3/3T/13, CZ.02.1.01/0.0/0.0/16_019/0000765 "Research Center for Informatics".

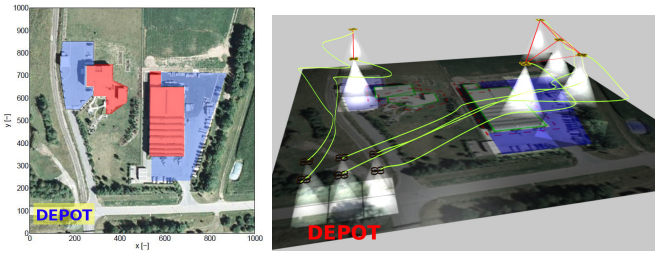


Fig. 1: Surveillance scenario. AoIs are depicted in blue.

Instead of using a global positioning system, MAVs can be stabilized relative to each other using a camera-based on-board localization system [2] to avoid mutual collisions. The direct vision-based localization system provides a fast and accurate distance measurement between team members, but neighboring MAVs need to be kept in constrained relative positions to achieve direct visibility.

The task of the mission planning is to find proper sensing locations for the MAVs above the given AoIs, and to find trajectories to reach these locations from a start location (a depot) satisfying localization and motion constraints. To achieve a reliable behavior of the system, the relative localization constraints have to be satisfied during the entire mission, as it cannot be ensured that the localization linkages will be established during the flight based on a GNSS system due to its insufficient precision, which motivated this research. An example of the surveillance mission with a team of UAVs covering the AoIs is depicted in Fig. 1.

In [1], this task of finding sensing locations and corresponding trajectories is formulated as a single optimization problem. Particle Swarm Optimization (PSO) is used in [1] to solve the problem. The disadvantage of this approach is that finding the appropriate sensing locations is time-consuming if the depot station is located far away from the AoIs. Moreover, the method [1] gets often stuck in local extremes in maps with obstacles.

In this paper, we present a system for mission planning of surveillance missions, where the goal is to find a suitable distribution of MAVs above the AoIs (sensing locations) that maximize the information gain and to find trajectories to these sensing locations. Finding the sensing locations cannot be decoupled from the trajectory computations, because not every sensing location is reachable from the depot by the MAVs whose motions are limited by the relative localization constraints.

We propose to solve this problem in a two-phase way:

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on March 13,2026 at 12:13:37 UTC from IEEE Xplore. Restrictions apply.

we first explore the state space using the Rapidly Exploring Random Tree (RRT) [3] planner. Unlike standard utilization of RRT, where the goal is specified explicitly, we describe the goal implicitly by a cost function. RRT is used to explore the space until a state with a non-zero cost function value is found. The found state then describes the sensing locations. During this exploration, motion model, as well as localization constraints, are considered, which ensures that the resulting sensing locations are reachable from the depot. Together with the sensing location, RRT also provides a feasible trajectory from the depot to the detected sensing locations. In the second phase, the sensing locations are optimized using Particle Swarm Optimization (PSO) [4]) to maximize the information gain. In comparison to [1], our approach also guarantees to find suitable sensing locations, but with a lower computational time and a shorter trajectory.

II. Related work

For a comprehensive review of works related to the multi-MAV surveillance that justifies the necessity of surveillance by a compact multi-MAV group see [1], where the given problem was introduced. In this section, this related work is updated with the latest available publications and with a description of works related to the addressed problem of RRT-based motion planning. Additionally, the state-of-the-art work [5] is used for a direct comparison with our solution that highlights contributions of this paper.

During a surveillance mission using MAVs, the AoI is typically observed by a downward facing camera. We can divide the surveillance missions to dynamic or static. In the dynamic surveillance robots or MAVs move on predefined trajectories, that are designed to cover the desired area maximally (e.g., [6]). This is related to the multi-robot coverage problem that has been traditionally studied with ground robots. In [7], the trajectories are computed using dynamic programming assuming that MAVs fly at fixed altitudes. In [8], the AoIs are first partitioned to a set of polygons, that are assigned to MAVs which cover them by a zig-zag pattern.

The static surveillance, which is considered in this paper, is achieved by hovering above the desired AoIs [9]. The main objective is to find the optimal placement of the MAVs to maximize the coverage which is related to the sensor placement problem [10]. As MAVs can fly in different altitudes, the area covered by the cameras increases with the increasing altitude, but the level of details decreases. It is, therefore, preferable to fly in a lower altitude and observe the region using multiple MAVs. In [11], the quality of sensing is measured by information per pixel and a cost function is defined for multiple down-facing cameras. The gradient of the cost function is used in a distributed control algorithm to position the MAVs to the optimal sensing location. The disadvantage of this system is that it cannot be used to fly from a distant depot to the sensing location as the control algorithm assumes that MAVs already hover above the AoI. Moreover, the utilization of this system is limited to the indoor systems only due to the dependence on the VICON localization system.

An important aspect of the surveillance using multiple MAVs is to avoid the collisions between the MAVs. The MAVs are usually navigated to the sensing locations using GPS [12] which might be too imprecise to avoid mutual collisions. Outdoor surveillance in GPS-denied environments using multiple MAVs was studied in [13]. First, a map of the environment is built using a single robot and the visual-SLAM technique. The distribution of MAVs is then optimized using Cognitive adaptive methodology. The MAVs are localized using the visual system relative to the map.

One of the main limitations of MAVs is their short flight time. In [14] a linear model of energy consumption is identified for MAVs. The coverage trajectories are then computed in two steps: first, the load-balancing problem is solved to fairly divides the area among MAVs, and then trajectories are computed using a modified Traveling Salesman Problem.

Another surveillance system for outdoor scenarios was proposed in our previous work [1]. The system utilizes an on-board visual-based relative localization [2] to ensure proper collision avoidance between team members. The optimization of the sensing location and computation of the feasible trajectories from the depot to these locations are solved in a single optimization. The optimization starts from depot station, and it ends when all AoIs are covered enough by the MAVs. Similarly to the approaches [9], [11], the cost function used in [1] evaluates the information gain. In comparison to [11], the approach [1] does not require MAVs to be located above AoIs. The single optimization approach can, however, be time-consuming if the depot is distant from the AoIs. In such a case, many optimization steps are required to find trajectories from the depot to the vicinity of AoIs.

To overcome this disadvantage, we propose in this paper to prepare the surveillance mission using a two-phase approach. First, trajectories of all MAVs from the depot are found close to the AoIs. The final position of MAVs above the AoIs are then obtained using an optimization approach. In both phases, the localization constraints and motion model of the MAVs need to be considered.

To ensure reachability of final MAV distribution from the depot station, the trajectories of individual MAVs need to be computed considering motion model of MAVs together with motion constraints caused by on-board localization system. Motion planning under these constraints can be solved using sampling-based techniques like Rapidly Exploring Random Trees [3]. RRT randomly samples configuration space of the robot in order to approximate its free regions by a tree rooted at the initial configuration. Moreover, the method can be easily extended for motion planning of multiple robots [15]. The authors of [16] present an RRT-based planner for finding collision-free trajectories for a team of UAVs. Although the planner does not consider the UAV motion model directly, the feasibility is provided by constraining the maximum turning angle and smoothing based on Dubins Path.

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on March 13,2026 at 12:13:37 UTC from IEEE Xplore. Restrictions apply.

III. Preliminaries

The configurations of all n MAVs in the global frame is described by the vector $q = (x_i, y_i, z_i, \varphi_i), i = 1, \ldots, n$. Each MAV is represented by its 3D position (x_i, y_i, z_i) and the heading φ_i. The configuration q satisfies the relative localization constraints if at least one neighbor of each MAV is in the distance $d_{\min}$ and $d_{\max}$. This constraint is necessary to enable the relative localization using on-board cameras [2]. The parameters $d_{\min}$ and $d_{\max}$ are determined by the employed localization system [17]. All possible configurations of the MAVs form the configuration (state) space $\mathcal{C}$. The configurations that satisfy the relative localization constraints and where the MAVs do not collide with each other or with the obstacles, form the set $\mathcal{C}_{free} \subseteq \mathcal{C}$.

We assume that MAVs can follow circular arcs, therefore the motion of individual MAVs is described as $\dot{x}_i = u_{s,i} \sin \varphi_i$, $\dot{y}_i = u_{s,i} \cos \varphi_i$, $\dot{z}_i = u_{z,i}$, and $\dot{\varphi}_i = u_{\phi,i}$, where $u_{s,i}$ is the forward speed, $u_{z,i}$ is the ascending speed and $u_{\phi,i}$ is the curvature. Each MAV is controlled by the tuple $(u_{s,i}, u_{z,i}, u_{\phi,i})$. Let $\mathcal{U}_i$ denote a set of control inputs for i-th MAV.

The map of the environment is known in advance, and it contains information about obstacles (no-flight zones) and regions to be observed. The map is represented by the matrix $A \in R^{a \times b}$ of size $a \times b$. The elements $A_{x,y}$ are either 0 or A_0, where A_0 denote cells to be observed, and zero elements are for regions not assigned to AoIs.

The quality of surveillance depends on the overall amount of information gained together by all MAVs. The AoIs can be observed either by one MAV flying at an optimal altitude S_{opt} specified for the mission, or by several MAVs flying at higher altitudes that are observing the same area. In the latter case, sensing cameras of MAVs are considered to provide less information, as they observe a larger area from a larger distance. The quality of surveillance at the configuration q is measured using the cost function $F(q)$ as

$$F(q) = \sum_{x=1}^{a} \sum_{y=1}^{b} \max \left(0, A_{x,y} - A_0 \sum_{i \in MAV(q)_{x,y}} \frac{S_{opt}}{S_i} \right), \quad (1)$$

where S_i is the area of the rectangle representing the part of the workspace that can be observed by i-th MAV, S_{opt} is the area of the region, which is observed by MAV flying at the "optimal" altitude, and $MAV(q)_{x,y}$ is a subset of all MAVs at configuration q that fully observe the cell $A_{x,y}$.

The optimal altitude S_{opt} is specified by an expert based on the particular application. We assume that MAV flying at the "optimal" altitude provides a sufficient amount of information on the part of AoI, which is covered by its surveillance sensor. An MAV at a lower altitude does not gain more information per square unit. On the contrary, we suppose that the information per square unit decreases linearly with increasing area S_i.

The required information from the particular element $A_{x,y}$ can be then obtained by a single MAV flying at the "optimal" altitude or cooperatively by a set of MAVs flying at higher altitudes. The sum of remainders (after subtraction of contributions of all MAVs) of the initial values A_0 assigned to each cell $A_{x,y}$ occupied by AoI is considered as the cost value of the particular solution. The cost function (1) is non-zero only around the AoIs, where MAVs observe at least part of the AoIs.

IV. Proposed surveillance mission planning

The task of the surveillance mission planning is to design suitable distribution of MAVs above the AoIs (sensing locations) that maximize the information gain and to find trajectories to these sensing locations. The second task is to provide trajectories for each MAV starting from the depot configuration $q_{init} \in \mathcal{C}_{free}$ and leading to the sensing locations. We propose to solve this problem by first exploring the space $\mathcal{C}$ using a sampling-based planner to find a suitable sensing location (i.e., we find q_{surv} for which $F(q_{surv})$ is minimized). The result of this first phase is the sensing location q_{surv} as well as feasible trajectories from q_{init} to q_{surv}.

The cost function (1), originally defined in [1] is hard to optimize, as it is decreasing only around the AoIs. In this paper, we utilize a smoother cost function

$$F'(q) = F(q) + \alpha \; d_{\mathrm{AoI}}(q), \quad (2)$$

where $d_{\mathrm{AoI}}(q)$ is the length of a 2D path from q to the center of the AoI. This adds information about the distance from AoI, obtained by the distance transform, to the cost function, which results in a more gradual optimization from the first iteration. Dijkstra algorithm is used to obtain the distance transform by running it with no stopping condition with the configuration inside AoI q_{goal} as the initial node. The distance is weighted by α as it is only helping information and must not influence the optimization of the original cost function.

V. Finding sensing locations using sampling-based planner

The sensing location q_{surv} is not defined explicitly, i.e., the exact positions of the MAVs above the AoIs where the information gain is maximized, are not known. They also cannot be derived based on the positions of the AoIs, as also the relative localization constraints have to be satisfied in q_{surv}. Moreover, the sensing location q_{surv} has to be reachable from q_{init}. The sensing location is defined implicitly, i.e., $F(q_{surv}) > 0$. Due to this reason, we formulate the search of q_{surv} as the search in the configuration space $\mathcal{C}$. Due to the high dimension of this space, we employ the sampling-based planning approach to find q_{surv}.

The configuration space is searched using the RRT sampling principle [3]. The algorithm builds a tree $\mathcal{T}$ rooted at q_{init}. The main loop of the method is listed in Alg. 1. In each iteration, a random sample q_{rand} is generated and it's nearest neighbor $q_{near} \in \mathcal{T}$ in the tree is found. The node q_{near} is then expanded towards q_{rand} and the new resulting configuration q_{new} is added to the tree. The localization constraints are checked for each of the resulting

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on March 13,2026 at 12:13:37 UTC from IEEE Xplore. Restrictions apply.

configurations. The tree is then expanded by such a feasible configuration q_{new}, that is nearest to q_{rand}. After the new configuration q_{new} is added to the tree, the cost function $F(q_{new})$ is evaluated. The search is terminated after a desired value of the cost function is reached, which means that a suitable sensing location was found. Alternatively, the search is terminated after a predefined number of planning iterations I_{max}.

The most time-consuming part of this RRT-based exploration is the expansion of the tree. The time complexity grows exponentially with number of MAVs. For each combination of control inputs $\mathcal{U}_1 \times \ldots \times \mathcal{U}_n$, motion model has to be evaluated, mutual collisions between the MAVs and the relative localization constraints have to be checked.

To speed up the expansion, we store the results of each expansion in a separate list. After a node $q_{near} \in \mathcal{T}$ is selected for the expansion for the first time, all combinations of control inputs $\mathcal{U}_1 \times \ldots \times \mathcal{U}_n$ are applied to the motion model and integrated over time Δt starting at the node q_{near}. The resulting configurations are sorted according to their increasing distance to q_{rand}, and their feasibility is examined in this order. The first configuration that satisfies feasibility constraints (i.e., it is collision-free, and it satisfies the localization constraints) is selected and added to the tree. The rest of the expanded configurations are stored in a list associated with q_{near} for future usage. Next time q_{near} is selected for the expansion, the next feasible configuration from the list is selected and added to the tree. This way, each combination of inputs is evaluated only once for each node, which significantly speeds up the planning process. Furthermore, if all combinations of control inputs are marked as used, the node is removed from the nearest-neighbor search to prevent its selection in future iterations.

The advantage of searching sensing locations using this RRT-based approach is that the provided trajectories are computed considering motion models and localization constraints, and it is therefore ensured, that they are feasible. An obvious disadvantage is that the configuration space is searched rather randomly and the search is not directed using the cost function. This means, that RRT with uniformly generated random samples has to explore the whole space until it reaches the vicinity of AoIs, where the cost function is non-zero. This is computationally demanding in the cases, where the depot is far away from the AoIs. To cope with this issue, we propose to guide the search towards the AoIs, which is described in the next sections.

A. Sampling along the guiding paths

To speed up the growth of the tree towards the AoIs, the sampling of the configuration space is guided using a geometric path in the workspace leading from the depot to the centers of the AoIs. The guiding principle proposed in this paper is an extension of the basic guiding technique [18]. We refer to this method as RRT-MPath in the rest of the paper. While [18] is guided only using one path, the proposed RRT-MPath is guided using multiple guiding paths as follows.

Let $P_i = (p_1, \ldots, p_{k_i})$, $p_i = (x, y, z)$ be a sequence of k_i 3D points representing the path in the workspace from the depot q_{init} to the center of the i-th AoI. Therefore, p_1 is the 3D position of the depot and p_{k_i} is the 3D position above the center of i-th AoI in the optimal altitude. The guiding path is computed without considering the heading of MAVs and also without considering the motion model and the relative localization constraints. The guiding path itself does not represent a feasible trajectory, but it represents a promising way around which a solution may be found. The guiding path can be computed, e.g., using A* in the grid-based representation of the environment.

Let $1 \leq v_i \leq k_i$ be the index of the active point of the guiding path P_i. The random samples are generated around the active point v_i. At the beginning of the algorithm, $v_i = 1, i = 1, \ldots, m$, where m is the number of AoIs. During the planning process, the virtual goal moves along the guiding path towards its end. In each iteration of RRT–MPath, the random sample q_{rand} is drawn uniformly from $\mathcal{C}$ with probability $(1 - p_{path})$, otherwise it is generated around the guiding paths as is described in the next subsection. If the tree $\mathcal{T}$ approaches the point v_i of the i-th path to a distance less than a predefined threshold d_{goal}, the active point v_i is moved to the next point on the path P_i. This ensures, that the configuration tree is attracted towards the guiding path and as soon as it reaches it, the virtual goal is moved forward, so that the tree is attracted to a new part of the guiding path.

Algorithm 1: RRT–MPath

Input: initial configuration $q_{init} \in \mathcal{C}$, guiding paths $P_1, \ldots, P_m$, probability of sampling around virtual goal p_{path}, cost function threshold $f_{desired}$, distance to reach the paths d_{goal}
Output: trajectory or failure

```
1  T.add(q_init); // new tree rooted at q_init
2  while iter++ < I_max do
3      if rand() < p_path then
4          q_rand = sample around guiding paths;
5      else
6          q_rand = random sample from C;
7      q_near=T.nearestNeighbor(q_rand);
8      q_new=expandConfiguration(q_near, q_rand);
9      if isFeasible(q_new) then
10         T.add(q_new);
11         if F(q_new) < f_desired then
12             return trajectory in tree from q_init to q_new;
13 return failure;
```

To attract the tree towards the guiding paths, the random samples need to be generated around the virtual goals. In the surveillance task with multiple AoIs, it may be required to split the MAVs into smaller teams, so that each team can visit different AoI. We propose two basic strategies to generate the random points $q_{rand} = (x_i, y_i, z_i, \varphi_i)$ along the guiding paths (the strategies are used in line 4 in Alg. 1). In both cases, the altitude z_i is generated uniformly from an allowed range of altitudes, and heading φ_i is generated randomly from $[0, 2\pi)$. The strategies differ in the way of generating

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on March 13,2026 at 12:13:37 UTC from IEEE Xplore. Restrictions apply.

positions x_i and y_i.

Algorithm 2: Sampling A

Input: configuration tree $\mathcal{T}$, number of MAVs n, guiding paths $P_1, \ldots, P_m$, distance to reach the active point d_{goal}

Output: q_{rand}

```
for i = 1, ..., n do
    r = index of a path, r = U(1, m);
    for j = length(P_r)...1 do
        d_j = distance from T to P_r[j];
        if d_j < d_goal then
            q_rand[i] = random sample around P_r[j];
            break;
```

Algorithm 3: Sampling B

Input: configuration tree $\mathcal{T}$, number of MAVs n, guiding paths $P_1, \ldots, P_m$, distance to reach the active point d_{goal}

Output: q_{rand}

```
r = index of a path, r = U(1, m);
for i = length(P_r)...1 do
    d_i = distance from T to P[i];
    if d_i < d_goal then
        q_rand[1...n] = random sample around P[i];
        break;
```

- In the first approach (denoted as Sampling A in the rest of the text), each MAVs is assigned to a random guiding path r in each iteration, and the position of the i-th MAV (x_i, y_i) is generated around the virtual goal v_r of the selected path. Let $(x_{v_r}, y_{v_r}) \in P_r$ denote 2D position of the virtual goal v_r on the randomly selected path r. Then, $x_i \sim N(x_{v_r}, \sigma^2)$ and similarly $y_i \sim N(y_{v_r}, \sigma^2)$. Since the guiding paths are selected randomly for each MAV, this sampling concept supports frequent splitting of the swarm. However, it is expected that the trajectory planning will need more time since MAVs closer to the AoI have to wait for slower MAVs behind them.
- In the second approach (Sampling B), a guiding path is selected randomly in each iteration and positions of all MAVs are generated around the virtual goal v of this path. The positions (x_i, y_i) are drawn using Normal distribution similarly as in the previous case, only the virtual goal is the same for all MAVs. As all MAVs are attracted to the same virtual goal v, this strategy should encourage fast movements of MAVs towards the goal. Contrary to the previous strategy, this approach does not support splitting of the MAVs to multiple groups.

If only one AoI needs to be surveyed, both strategies provide the same results.

VI. Coverage optimization

The search of the sensing location q_{surv} realized by the above-mentioned method terminates if a configuration q_{surv} such that $F(q_{surv}) > 0$ is found. This means, that the sampling-based planner found a configuration where all AoIs are visible from the MAVs, but it is not ensured, that the sensing location q_{surv} is optimal. The sensing location q_{surv} is further optimized by the PSO technique [4]. PSO is an evolutionary-based method, that maintains a population of k particles. Each particle represents a candidate solution, i.e., the configuration q describing the position of all n MAVs. The particles move in the $4m$-dimensional search space, as the configuration of each MAV is described by four parameters. Each particle $q_i, i = 1, \ldots, k$ has a velocity v_i in the search space. In each iteration k of PSO, velocities and positions of the $i-th$ particles are updated as:

$$\begin{aligned} v_i(k) &= v_i(k\text{–}1) + r_1\varphi_1(b_i\text{–}q_i(k\text{–}1)) + r_2\varphi_2(B\text{–}q_i(k\text{–}1)) \\ q_i(k) &= q_i(k\text{–}1) + v_i(k), \end{aligned} \tag{3}$$

where φ_1 and φ_2 influence the convergence of the search (we refer to [4] for details), r_1 and r_2 are vectors of random numbers from the uniform distribution $U(0, 1)$. B is the "best" particle in the whole history of the optimization and b_i is the best position of the i-th particle in the history. The global and local best particles B and b_i respectively are determined using the cost function $F(q)$ in each iteration.

The basic PSO rule (3) updates the position of the particles without considering the motion model, so the transition from $q_i(k-1)$ to $q_i(k)$ might not be feasible. To ensure a proper transition, the new position $q_i(k)$ has to be updated considering the motion model and the localization constraints. After the PSO rule is applied to get new $q_i(k)$, various control inputs $u \in \mathcal{U}_1 \times \ldots \times \mathcal{U}_n$ are applied to each MAV using the motion model, which will lead to new configurations $q_i^u(k) \in \mathcal{C}$. The final position of the particle is set to such $q_i^u(k) \in \mathcal{C}_{free}$, where the localization constraints are satisfied and where the MAVs do not collide with each other or with the obstacles. This ensures, that each transition from $q_i(k-1)$ to $q_i(k) = q_i^u(k)$ is feasible, as it is obtained using the forward motion model and because $q_i^u(k) \in \mathcal{C}_{free}$. In this approach, the PSO mainly controls the direction of the particles in order to minimize the cost function, while the real movements of the particles are realized using the motion model. We refer to [1] about technical details.

The optimization starts with $q(1) = q_{surv}$, i.e., with MAVs located at the sensing location q_{surv}, $v(0)$ is set to random numbers from $U(0, 1)$. After the predefined number of iterations (or when the position of the best particle B does not change for a specific number of iterations), the final solution is represented by B.

VII. Verification in simulation

A. Scenario with single AoI

The proposed motion planning method has been verified in simulation and compared to the RRT planner and the PSO-based [1] planner. The PSO-based planner [1] is run with 20 particles and 400 iterations. The simulation experiments have been performed with four MAVs. Each MAV is controlled using $|\mathcal{U}_i| = 4$ combinations of control inputs leading to a

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on March 13,2026 at 12:13:37 UTC from IEEE Xplore. Restrictions apply.

total of 256 combinations of control inputs. The experiments have been performed on Intel Core i7-860@2.8 GHz.

The methods have been tested in three environments with various complexity. The size of each environment is 1000×1000 m, the size of each MAV is 0.5 m and localization constraints are $d_{min} = 3$ m and $d_{max} = 20$ m. A single obstacle is in the Simple map. The map Maze contains complex obstacles which requires the team to maneuver between them, but still with enough space between the obstacles. The Narrow map contains a narrow passage. The task of the planners is to find trajectories for four MAVs from a depot station to the vicinity of the AoI. The planners are terminated if all MAVs are above the AoI. Examples of found trajectories are shown in Fig. 2.

The average results from 50 trials are shown in Tab. I. The success rate is computed as the number of trials where the AoI is reached over the total 50 trials. All planners are able to find a solution in the Simple map (success rate is 100 %). The RRT–MPath method is significantly faster than RRT as it samples the configuration space along the guiding path contrary to RRT, that explores the whole configuration space. This is also indicated by the number of iterations, that is higher for RRT. RRT–MPath is superior also in Maze and Narrow environments. The slowest method is PSO-based planner. Although it has a similar amount of iterations as RRT–MPath, it is significantly slower, which is caused by a frequent violation of localization constraints, which requires to restart velocities of the PSO particles [1].

B. The scenario with multiple AoIs

In the second experiment, the effect of sampling strategies described in Section V-A is verified in the scenario with four MAVs and two AoIs. To enable the coverage of two AoIs using four MAVs, the localization constraints were simplified, so each MAV has at least one neighbor in the distance from d_{min} to d_{max}. The planner was terminated after $I_{max} = 2000$ iterations.

The average results over 20 trials are shown in Tab. II, where the quality of coverage is denoted in the row AoI covered, which is the sum of the percentage of coverage of each AoI. For two AoIs, the value 2 denotes that both AoIs are fully covered, and 1 means that only 50 % of the areas was covered. The best coverage is achieved by the automatic splitting of the team using Sampling A. Sampling B generates random samples for all MAVs along the same guiding path, which does not support the team splitting.

The coverage performance of RRT–MPath and PSO in a map with 4 AoIs which have to be covered by a team of 4 UAVs is compared in Tab. III. The PSO found a solution faster, however, only one AoI was covered in each of 100 trials.

VIII. Deployment on physical MAVs

The developed method has been verified with 3 physical MAVs. The MAVs are built on the DJI F550 frame equipped with PX4 Pixhawk Flight Control Unit, laser rangefinder Garmin LIDAR-Lite v3 and relative localization system [19] consisting of a circular pattern which is detected by Mobius ActionCam. The trajectory of circular arc segments provided by the proposed system is realized using on-board MPC controller in an open environment of $80 \times 40\,m$ with simulated obstacles. The top view of the performed experiment is depicted in Fig. 3.

The core part of the experiment is to verify the feasibility of the planned swarm trajectory in terms of relative localization. The visibility of the circular pattern has to be ensured during the whole flight which is accomplished by maintaining a predefined spacing of the neighbor MAVs in addition to following the planned MAV heading to keep the circular pattern in the camera Field of View. A distance range $d = [7, 9]\,m$ was chosen with respect to the size of the circular pattern, the camera focal length, and the visibility conditions. The maximum allowed deviation of heading was fixed at $60°$ with $0°$ deviation corresponding to the circular pattern being in the horizontal center of the camera image. The nearest neighbor distances can be seen in Fig. 4. The heading deviation from the optimal yaw angle of all MAVs is depicted in Fig. 5.

IX. Conclusion

A novel motion planning algorithm has been proposed to solve a cooperative surveillance scenario by multiple MAVs. In this task, MAVs have to observe desired areas of interest, which requires to find appropriate sensing locations together with feasible trajectories from a depot to reach the locations. The MAVs are localized using an on-board localization system, which limits relative motions of MAVs in order to ensure their mutual visibility. We propose to find the sensing locations using sampling-based motion planning and optimize them further by the Particle Swarm Optimization. As the sensing locations are searched considering the localization constraints as well as motion model of the MAVs it is ensured that the found sensing locations are reachable from the depot while satisfying all the constraints.

References

[1] M. Saska, J. Chudoba, L. Přeučil, J. Thomas, G. Loianno, A. Třešňák, V. Vonásek, and V. Kumar, "Autonomous Deployment of Swarms of Micro-Aerial Vehicles in Cooperative Surveillance," in *Proceedings of 2014 International Conference on Unmanned Aircraft Systems (ICUAS)*, vol. 1, 2014, pp. 584–595.

[2] T. Krajník, M. Nitsche, J. Faigl, P. Vaněk, M. Saska, L. Přeučil, T. Duckett, and M. Mejail, "A practical multirobot localization system," *Journal of Intelligent Robotics Systems*, vol. 76, no. 3-4, pp. 539–562, 2014.

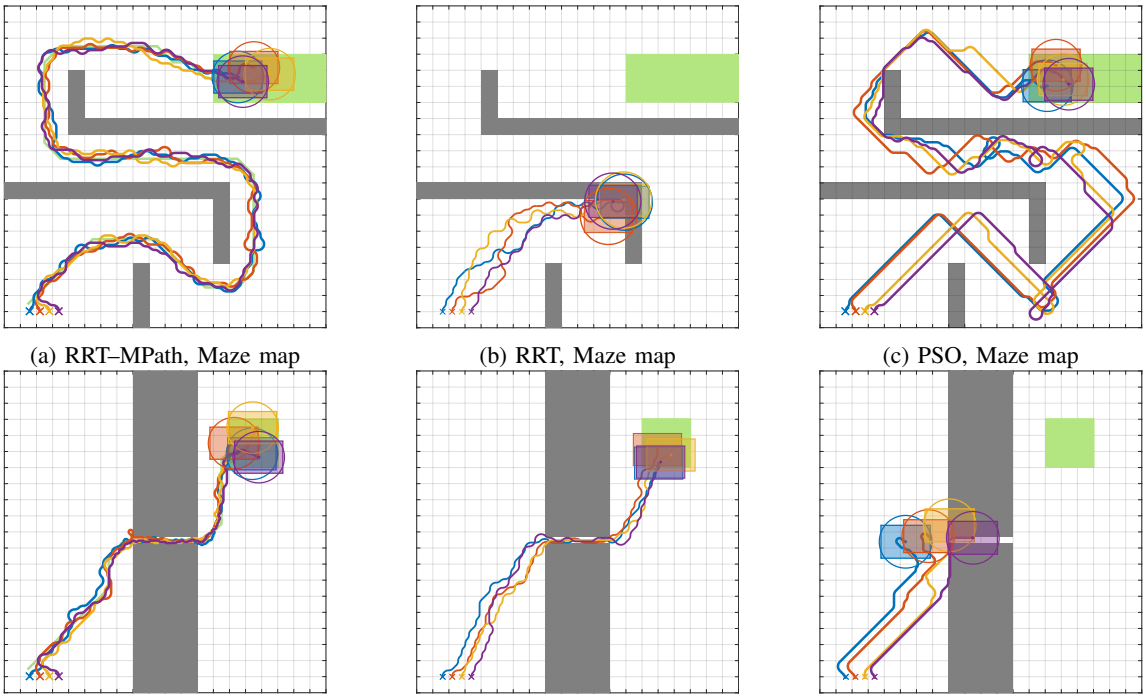
[3] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," 1998, tR 98-11.

[4] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *IEEE International conference on Neural Networks*, 1995.

[5] M. Saska, T. Baca, J. Thomas, J. Chudoba, L. Preucil, T. Krajnik, J. Faigl, G. Loianno, and V. Kumar, "System for deployment of groups of unmanned micro aerial vehicles in gps-denied environments using onboard visual relative localization," *Autonomous Robots*, vol. 41, no. 4, pp. 919–944, 2017.

[6] S. Winkler, M. Buschmann, L. Kruger, H. w. Schulz, and P. Vorsmann, "Automav - micro aerial vehicles for airport surveillance," in *TENCON 2005 - 2005 IEEE Region 10 Conference*, Nov 2005, pp. 1–6.

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on March 13,2026 at 12:13:37 UTC from IEEE Xplore. Restrictions apply.



(a) RRT–MPath, Maze map (b) RRT, Maze map (c) PSO, Maze map

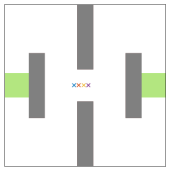
(d) RRT–MPath, Narrow passage map (e) RRT, Narrow passage map (f) PSO, Narrow passage map

Fig. 2: Examples of trajectories for surveillance with four MAVs.

TABLE I: Comparison of planning methods on different maps. N/A denotes situation where planner failed to find any solution.

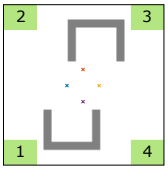
Map		RRT–MPath		RRT		PSO	
		Mean	Dev	Mean	Dev	Mean	Dev
Simple	**Success rate**	100 %		100 %		100 %	
	Runtime [s]	5.05	5.41	23.8	16.63	186.72	25.6
	Iterations	203.4	59.23	913.68	378.68	250.38	33.14
	Trajectory length	1182.23	27.02	1125.34	47.45	1445.94	199.65
Maze	**Success rate**	98 %		0 %		6 %	
	Runtime [s]	12.59	1.96	N/A		322.45	77.56
	Iterations	467.49	67.83	N/A		400	95.96
	Trajectory length	2823.38	400.78	N/A		3631.67	871.41
Narrow	**Success rate**	82 %		38 %		0 %	
	Runtime [s]	11.31	14.09	108.67	78.43	N/A	
	Iterations	348.95	167.78	1788.84	1026.93	N/A	
	Trajectory length	1141.81	443.77	1151.36	565.69	N/A	

TABLE II: Performance of sampling with multiple AoIs (green). The depot is in the center of the map.



	A	B
AoI covered	1.64	1.31
Tree size	1647	258
Runtime [s]	117.42	7.32

TABLE III: Comparison of RRT-Path and PSO in coverage performance with 4 AoIs (green). The depot is in the center of the map. The experiment was run 100 times.



	RRT-MPath	PSO
AoI #1 reached	33	0
AoI #2 reached	66	0
AoI #3 reached	96	0
AoI #4 reached	20	100
Path length [m]	89.60	129.52
Runtime [s]	779.40	38.46

[7] A. Ahmadzadeh, J. Keller, G. Pappas, A. Jadbabaie, and V. Kumar, "An optimization-based approach to time-critical cooperative surveillance and coverage with uavs," in *Experimental Robotics*. Springer, 2008, pp. 491–500.

[8] I. Maza and A. Ollero, "Multiple uav cooperative searching operation using polygon area decomposition and efficient coverage algorithms," in *Distributed Autonomous Robotic Systems 6*. Springer, 2007, pp. 221–230.

[9] S. Papatheodorou, A. Tzes, and Y. Stergiopoulos, "Collaborative visual area coverage using unmanned aerial vehicles," *CoRR*, vol. abs/1612.02065, 2016.

[10] L. C. A. Pimenta, V. Kumar, R. C. Mesquita, and G. A. S. Pereira, "Sensing and coverage for a network of heterogeneous robots," in *2008 47th IEEE Conference on Decision and Control*, 2008, pp. 3947–3952.

[11] M. Schwager, B. J. Julian, and D. Rus, "Optimal coverage for multiple hovering robots with downward facing cameras," in *IEEE International Conference on Robotics and Automation*, 2009.

[12] A. Renzaglia, L. Doitsidis, A. Martinelli, and E. B. Kosmatopoulos, "Adaptive-based distributed cooperative multi-robot coverage," in *Proceedings of the 2011 American Control Conference*, 2011, pp. 468–473.

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on March 13,2026 at 12:13:37 UTC from IEEE Xplore. Restrictions apply.

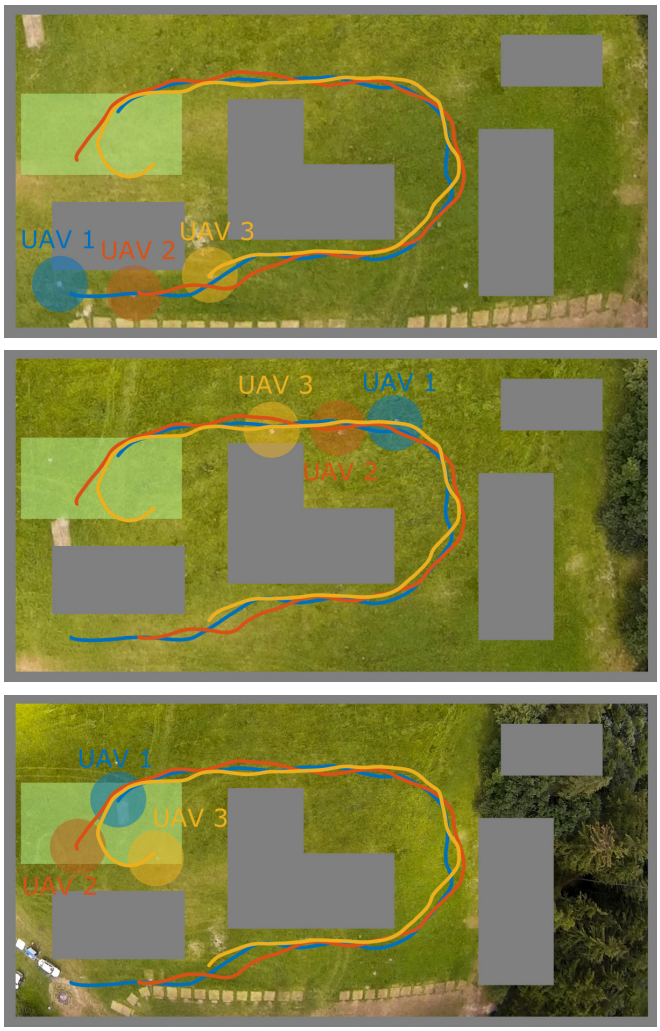


Fig. 3: The snapshots of trajectory of MAV swarm following the trajectory to the sensing locations in the real environment with simulated obstacles in gray and AoI in green color.

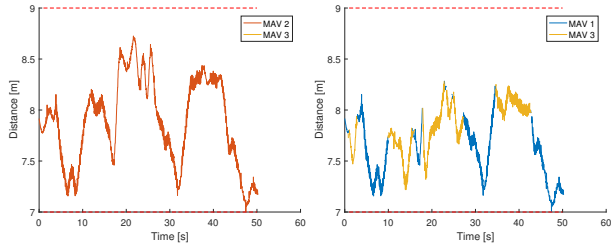


Fig. 4: Distance to the nearest neighbor of MAV1 (left) and MAV2 (right) with the line color representing which MAV is the nearest neighbor. The distances were obtained from positions reported by MAV odometry during a flight experiment.

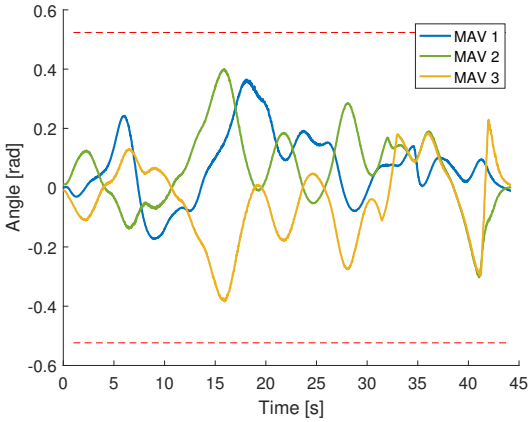


Fig. 5: The yaw deviation from the ideal MAV heading, with zero deviation meaning, that the neighbor MAV is in the center of the field of view of the MAV. The red dashed lines represent the constraints of the MAV field of view.

[13] L. Doitsidis, A. Renzaglia, S. Weiss, E. Kosmatopoulos, D. Scaramuzza, and R. Siegwart, "3d surveillance coverage using maps extracted by a monocular slam algorithm," in *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2011, pp. 1661–1667.

[14] J. Modares, F. Ghanei, N. Mastronarde, and K. Dantu, "Ub-anc planner: Energy efficient coverage path planning with multiple drones," *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6182–6189, 2017.

[15] V. Vonásek, M. Saska, and L. Přeučil, "Motion Planning for a Cable Driven Parallel Multiple Manipulator Emulating a Swarm of MAVs," in *ROBOT MOTION AND CONTROL 2013*. Pistacaway, NJ: IEEE Robotics and Automation Society, 2013, pp. 13–18.

[16] W. Zu, G. Fan, Y. Gao, Y. Ma, H. Zhang, and H. Zeng, "Multi-uavs cooperative path planning method based on improved rrt algorithm," in *2018 IEEE International Conference on Mechatronics and Automation (ICMA)*, 2018, pp. 1563–1567.

[17] J. Faigl, T. Krajnik, J. Chudoba, L. Preucil, and M. Saska, "Low-cost embedded system for relative localization in robotic swarms," in *IEEE International Conference on Robotics and Automation*, 2013.

[18] V. Vonásek, J. Faigl, T. Krajník, and L. Přeučil, "RRT-Path: a guided Rapidly exploring Random Tree," in *Robot Motion and Control 2009*. Heidelberg: Springer, 2009, pp. 307–316.

[19] T. Krajník, M. Nitsche, J. Faigl, P. Vaněk, M. Saska, L. Přeučil, T. Duckett, and M. Mejail, "A practical multirobot localization system," *J. Intell. Robotics Syst.*, vol. 76, no. 3-4, pp. 539–562, Dec. 2014.

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on March 13,2026 at 12:13:37 UTC from IEEE Xplore. Restrictions apply.